

# LaGraph: Laplacian-Guided Graph Learning for Time Series Anomaly Detection

Shicong Zeng<sup>✉</sup>, Guoqing Chao<sup>✉</sup>, Junquan Wei<sup>✉</sup>, Yanwei Yu<sup>✉</sup>, *Member, IEEE*, Zhijin Wang<sup>✉</sup>, *Member, IEEE*, and Dianhui Chu<sup>✉</sup>

**Abstract**—Time series anomaly detection is crucial in fields such as industrial monitoring, financial risk management, and network security. Graph Neural Networks (GNNs) have demonstrated strong capabilities in capturing multivariate dependencies. However, existing methods often fail to adequately account for the temporal proximity between adjacent time points and are susceptible to the influence of weak or noisy connections during graph-based representation learning. To address these challenges, we propose LaGraph, a novel framework that integrates GNNs with a mask-optimized attention mechanism. Specifically, LaGraph decomposes input sequences into stable and trend components using an Expert Decomposition Block. The trend component is processed via a Multi-layer Convolution Block, while the stable component is modeled with a Proximity-enhanced Graph Convolutional Network that incorporates a Laplacian kernel to capture local temporal dependencies. Additionally, a Mask-optimized Multi-head Attention Block, based on the Straight-Through Estimator (STE), mitigates the negative effects of less informative edges, enhancing both representation quality and reconstruction performance. Extensive experiments on five real-world benchmark datasets demonstrate that LaGraph consistently outperforms state-of-the-art methods, verifying its effectiveness and superiority for time series anomaly detection.

**Index Terms**—Time series anomaly detection, temporal proximity, Laplacian kernel, graph neural networks, mask-optimized attention.

## I. INTRODUCTION

**T**IME series anomaly detection plays a crucial role in various real-world applications, including industrial fault diagnosis, financial risk management, healthcare monitoring, and intelligent transportation systems [1], [2], [3], [4]. Its goal

is to identify data points that significantly deviate from normal temporal patterns. However, this task remains challenging due to the scarcity of anomalies, the diverse nature of anomaly types, complex temporal dependencies, and the difficulty of modeling hidden relationships among multiple variables.

The rarity of anomalous events, combined with the high cost and the need for expert knowledge during labeling, poses significant challenges in constructing large-scale labeled datasets [5], [6], [7]. Consequently, supervised methods are often impractical in real-world scenarios. In contrast, unsupervised methods have garnered increasing attention in recent years, as they focus on modeling normal patterns and detecting anomalies based on deviations from these patterns, without requiring labeled anomaly data [8].

Existing time series anomaly detection methods can be broadly categorized into statistical methods, traditional machine learning methods, and deep learning-based methods [9], [10]. Statistical methods, such as ARIMA [11], seasonal decomposition [12], and CUSUM [13], typically rely on assumptions of stationarity or known data distributions. While effective for simple and regular patterns, their performance often deteriorates when applied to nonlinear, non-stationary, or high-dimensional time series. Traditional machine learning methods, including isolation forests [14], support vector machines [15], and k-nearest neighbors [16], depend on hand-crafted features and fixed decision boundaries, making them sensitive to hyperparameter choices and limited in modeling long-range temporal dependencies.

Deep learning methods have shown superiority to model complex temporal patterns. Among them, three main paradigms are commonly adopted. Prediction-based methods [17], [18], [19] train models to forecast future values and identify anomalies based on prediction errors. Reconstruction-based methods [20], [21], [22], [23] aim to learn normal time series patterns and detect anomalies by measuring the discrepancy between the original inputs and their reconstructed counterparts. Latent representation-based methods differentiate normal and abnormal behaviors through discriminative or contrastive learning objectives to learn effective representation [24], [25].

Among deep learning approaches for time series anomaly detection, reconstruction-based methods have gained considerable attention, particularly in unsupervised settings. These methods aim to model the underlying structure and temporal patterns of normal data and reconstruct the input sequence. The core idea is that inputs deviating from learned normal behavior will

Received 1 August 2025; revised 26 January 2026; accepted 14 February 2026. Date of publication 17 February 2026; date of current version 9 April 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62276079, in part by the Young Teacher Development Fund of Harbin Institute of Technology under Grant IDGA10002071, in part by the Research and Innovation Foundation of Harbin Institute of Technology under Grant IDGAZMZ00210325, and in part by the Special Funding Program of Shandong Taishan Scholars Project. Recommended for acceptance by S. Whang. (Corresponding author: Guoqing Chao.)

Shicong Zeng, Guoqing Chao, Junquan Wei, and Dianhui Chu are with the School of Computer Sciences and Technology, Harbin Institute of Technology, Weihai 264209, China (e-mail: guoqingchao@hit.edu.cn).

Yanwei Yu is with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao 266005, China.

Zhijin Wang is with the College of Computer Engineering, Jimei University, Xiamen 361021, China.

To promote reproducibility and support future research, we have publicly released the full implementation at <https://github.com/hit-zsc/LaGraph>.

Digital Object Identifier 10.1109/TKDE.2026.3665696

result in higher reconstruction errors, which can be used as effective signals for anomaly detection. Since these methods do not require labeled anomalies, they are suitable for real-world scenarios where annotated data is scarce or unavailable. In addition, reconstruction-based methods can capture complex temporal dependencies and provide an interpretable anomaly scoring mechanism through reconstruction loss. Despite these advantages, reconstruction-based methods still face significant challenges. Their effectiveness depends largely on the model's ability to accurately capture normal patterns without overgeneralizing to unseen abnormal inputs. Thus, designing robust model architectures and loss functions is crucial to guarantee reliable performance in diverse and complex time series scenarios.

Graph Neural Networks (GNNs) [26] have demonstrated strong performance in modeling structural dependencies within multivariate time series, making them particularly suitable for reconstruction-based anomaly detection. By incorporating graph structures, GNNs explicitly capture relationships and structural similarities among variables, which is crucial for accurate sequence reconstruction. However, conventional GNNs often overlook an essential characteristic of time series data: temporal proximity, which refers to the intrinsic similarity between adjacent time points. Ignoring this property limits the model's ability to capture fine-grained temporal dynamics, which is critical for precise anomaly detection. Moreover, the learned graphs in existing methods are often dense and noisy; spurious edges can propagate noise through message passing and destabilize reconstruction, especially under non-stationary drifts and level shifts.

To address these issues, we propose a Laplacian kernel-enhanced graph convolutional network that integrates temporal awareness into the graph learning process. The proposed method introduces an explicit temporal proximity prior and performs noise-aware graph sparsification during message passing, avoiding reliance on similarity-driven dense temporal graphs. Specifically, we construct a time-aware graph using a Laplacian kernel to explicitly emphasize the temporal proximity between adjacent time points. This kernel serves as a prior to modulating edge weights in the graph convolution, enabling the model to capture locally coherent temporal structures. Furthermore, to suppress the influence of weakly connected nodes and noisy edges, we introduce a mask-optimized multi-head attention mechanism based on the Straight-Through Estimator (STE) [27]. This mechanism adaptively filters out noisy connections, allowing the model to focus more on the meaningful relationships or structures in the data, which improves anomaly detection.

Our proposed model, **LaGraph**, consists of four core modules. First, the Expert Decomposition Block separates the raw time series into stable and trend components, enabling specialized modeling of distinct temporal dynamics and reducing the contamination of reconstruction by non-stationary components. Second, the trend component is passed through a Multi-layer Convolution Block to extract global temporal patterns. Third, the stable component is processed by the Proximity-enhanced Graph Convolutional Network, which captures fine-grained temporal correlations using a time-aware graph structure that emphasizes temporal proximity between time points. Finally, the

Mask-optimized Multi-head Attention Block integrates long-range dependencies while suppressing irrelevant or noisy graph connections, thereby improving reconstruction quality. By jointly modeling structural and temporal similarities and explicitly addressing noise in the learned temporal graph, the proposed framework enhances both the expressiveness of the reconstruction process and the reliability of anomaly detection.

The main contributions of this paper are summarized as follows:

- We propose an Expert Decomposition Block that separates the input time series into stable and trend components, reducing the contamination of reconstruction by non-stationary components and enabling targeted modeling of distinct temporal dynamics. This design improves robustness by isolating trend-induced variations and reconstructing more stationary residual patterns.
- We design a Proximity-enhanced Graph Convolutional Network that incorporates a Laplacian kernel to emphasize temporal proximity between adjacent time points, providing a principled temporal prior and thereby enhancing the model's ability to capture fine-grained local temporal dependencies.
- We propose a Mask-optimized Multi-head Attention Block based on the STE to adaptively suppress weak and noisy connections in the temporal graph structure, leading to more reliable feature aggregation and improving both reconstruction accuracy and anomaly detection robustness.
- Extensive experiments on five widely used benchmark datasets demonstrate that our method achieves state-of-the-art performance compared with strong recent baselines, verifying its effectiveness and superiority.

## II. RELATED WORKS

### A. Time Series Anomaly Detection

Traditional time series anomaly detection methods can be broadly classified into statistical modeling and classical machine learning techniques. Statistical approaches typically decompose time series into trend, seasonal, and residual components to detect anomalies, whereas machine learning methods are commonly based on clustering, classification, or density estimation techniques. Despite their success in simple or low-dimensional cases, these methods generally fail to scale effectively to high-dimensional and dynamically evolving multivariate time series.

In recent years, deep learning has emerged as a powerful paradigm for time series anomaly detection. By directly learning temporal structures and nonlinear patterns from raw data, these models eliminate the need for manual feature engineering and offer enhanced modeling flexibility. Prominent architectures include recurrent neural networks (e.g., RNNs, LSTMs) [28] for modeling temporal dependencies, Transformer-based models [29] for capturing long-range interactions via attention mechanisms, and GNNs for encoding inter-variable relationships in multivariate settings. Compared with traditional methods, deep learning approaches have exhibited improved accuracy, adaptability, and scalability, and have thus become a primary focus of recent research.

### B. Graph Neural Network for Time Series

GNNs, a class of deep learning models designed for non-Euclidean structured data, have emerged as a powerful paradigm for modeling complex dependencies among multivariate time series. Unlike traditional sequence models that treat each variable as an independent channel or rely solely on fixed temporal orderings, GNNs employ an explicit graph-based framework to model the inter-variable relationships, both spatial and temporal, which are often dynamic. This framework enables the model to learn richer representations of temporal and structural patterns, enhancing both its expressiveness and generalization ability.

Over the years, several variants of GNNs have been proposed to enhance their capacity for learning from graph-structured data. For instance, Graph Convolutional Networks (GCNs) introduced by Kipf and Welling [30] focus on spectral-based graph convolutions to aggregate node features, while Graph Attention Networks (GATs) [31] leverage attention mechanisms to dynamically weight the importance of neighboring nodes. Additionally, GraphSAGE [32] employs inductive learning techniques that allow for scalability to large graphs, and Graph Isomorphism Networks (GINs) [33] enhance expressive power by using more powerful aggregation functions. These advancements have greatly expanded the applicability of GNNs to a variety of tasks, such as node classification, link prediction, and graph generation.

In time series anomaly detection, GNNs play a crucial role in capturing the complex structural dependencies among variables [34], which are essential for identifying subtle deviations from normal patterns [35], [36], [37], [38], [39], [40], [41]. These dependencies may include temporal correlations within individual time series, as well as interactions across multiple dimensions or sensor channels. The graph structure can either be constructed based on prior knowledge, such as physical topology or domain-specific layouts, resulting in static graphs, or learned adaptively from the data to reflect latent, time-varying relationships. By integrating these dynamic graph structures with temporal modeling components, such as recurrent neural networks or attention mechanisms, GNNs form spatio-temporal architectures that can simultaneously model temporal evolution and inter-variable interactions, which are critical for robust anomaly detection in time series data.

Beyond Euclidean GNNs, recent studies have explored dynamic graph learning in Riemannian and hyperbolic spaces [42], [43]. By embedding nodes on curved manifolds and using manifold-specific operations, such as exponential and logarithmic maps, these models are particularly effective when the underlying relations exhibit strong hierarchy or tree-like structure. In LaGraph, however, graphs are constructed within short sliding windows to model local temporal interactions, where relational patterns are largely non-hierarchical. In this regime, a Euclidean GCN provides a favorable accuracy–efficiency trade-off while avoiding the additional computational overhead of manifold-based layers. Importantly, our key components are geometry-agnostic; thus, replacing the Euclidean GCN with a Riemannian or hyperbolic variant is a natural extension that we leave for future work.

### C. Transformer for Time Series

The Transformer architecture, first introduced by Vaswani et al. [29], has become a foundational deep learning model in time series analysis due to its strong modeling capacity and efficient parallel computation [44], [45], [46], [47], [48], [49], [50], [51]. However, conventional Transformer models encounter difficulties in processing long sequences, particularly in capturing long-range dependencies. To address these limitations, various architectural enhancements have been proposed, including improved temporal encoding strategies and optimized attention mechanisms, which reduce computational complexity and enhance efficiency. These innovations allow Transformer-based models to capture complex temporal dynamics and distant dependencies more effectively, thereby mitigating the shortcomings of earlier approaches.

For instance, Zhou [52] introduced the sparse attention mechanism, effectively addressing the quadratic time complexity problem in Transformer models. Wu [53] replaced the self-attention mechanism with an auto-correlation mechanism, achieving promising results. Zhou [54] proposed a frequency-enhanced Transformer that efficiently captures temporal dynamics while reducing computational complexity, further enhancing the model’s scalability and performance.

## III. METHODOLOGY

### A. Problem Statement

Given a multivariate time series  $\mathbf{X} \in \mathbb{R}^{T \times N}$ , where  $T$  denotes the number of time steps and  $N$  represents the number of variables or sensor channels, the objective is to learn a function  $f: \mathbb{R}^{T \times N} \rightarrow \{0, 1\}^T$  that maps the input sequence to a binary label sequence  $y \in \{0, 1\}^T$ . Each element  $y_t$  indicates whether the time step  $t$  corresponds to an anomaly ( $y_t = 1$ ) or not ( $y_t = 0$ ).

This task is formally defined as a sequence-level anomaly detection problem, where the aim is to identify time points exhibiting abnormal behavior in any of the observed variables. The primary challenge lies in effectively modeling both the temporal dependencies and the complex inter-variable correlations to detect subtle or sparse anomalies.

### B. Framework

Figure 1 illustrates the overall framework of the proposed method, which comprises four main modules: (1) the Expert Decomposition Block, responsible for decomposing the input time series into stable and trend components; (2) the Multi-layer Convolution Block, which applies lightweight convolutional operations to extract temporal patterns from the trend component; (3) the Proximity-enhanced Graph Convolutional Network, where the graph structure is refined using a Laplacian kernel prior to enhance sensitivity to adjacent time steps; (4) the Mask-optimized Multi-head Attention Block, which incorporates a mask matrix generated via the STE to suppress weak connections between distant time points in the sequence.



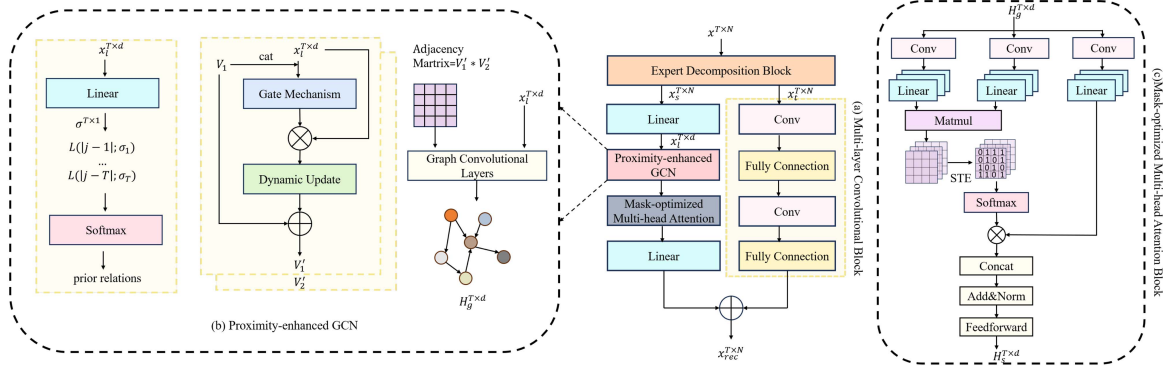


Fig. 1. The framework of LaGraph is illustrated. (a) shows the process of the Multi-layer Convolution Block, (b) depicts the Proximity-enhanced GCN process, and (c) explains the operation of the Mask-optimized Multi-head Attention Block. Expert Decomposition Block will be detailed in Fig. 2.

### C. Expert Decomposition Block

To address the issue of significant performance variation across different datasets in decomposition modules, we propose the Expert Decomposition Block. Specifically, we define  $M$  moving average experts, each corresponding to an averaging operation with a distinct convolutional kernel size. A gating network is utilized to generate a set of weights for these experts. The trend component is derived by performing a weighted fusion of the expert outputs, and the stable component is obtained by subtracting the trend component from the original input. Given an input  $\mathbf{X} \in \mathbb{R}^{T \times N}$ , this process can be formulated as follows:

$$\mathbf{S}_i = \text{MA}_i(\mathbf{X}) \in \mathbb{R}^{T \times N}, \quad i = 1, 2, \dots, M, \quad (1)$$

$$\mathbf{g} = \text{Gate}(\mathbf{X}) \in \mathbb{R}^M, \quad (2)$$

$$\boldsymbol{\alpha} = \text{Softmax}(\mathbf{g}) = [\alpha_1, \alpha_2, \dots, \alpha_M]^\top, \quad (3)$$

$$\mathbf{X}_t = \sum_{i=1}^M \alpha_i \mathbf{S}_i, \quad (4)$$

$$\mathbf{X}_s = \mathbf{X} - \mathbf{X}_t. \quad (5)$$

Here,  $\text{MA}_i(\cdot)$  is the moving average operator with kernel size  $k_i$ , implemented using symmetric edge padding followed by a 1D average-pooling layer. This design ensures that  $\text{MA}_i(\mathbf{X})$  preserves the original temporal length  $T$ , and each expert output  $\mathbf{S}_i$  corresponds to a trend component extracted at a different smoothing scale. The gating network  $\text{Gate}(\cdot)$  first aggregates the input through global average pooling along the temporal dimension and then applies a linear projection to produce an  $M$ -dimensional logit vector  $\mathbf{g}$ . After Softmax normalization, the resulting coefficients  $\boldsymbol{\alpha} = [\alpha_1, \dots, \alpha_M]^\top$  satisfy  $\alpha_i \geq 0$  and  $\sum_{i=1}^M \alpha_i = 1$ , and therefore can be interpreted as the relative contribution of each expert. The final trend component  $\mathbf{X}_t$  is computed as a weighted sum of the expert outputs, and the stable component  $\mathbf{X}_s \in \mathbb{R}^{T \times N}$  is obtained by subtracting  $\mathbf{X}_t$  from the original input.

### D. Multi-Layer Convolution Block

To effectively leverage the latent patterns within the trend, we apply multiple stacked convolutional layers, each followed by a fully connected layer. Formally, for an input  $\mathbf{X}_t \in \mathbb{R}^{T \times N}$ , the

process is expressed as:

$$\mathbf{H}_t = \text{FC}(\text{Conv}(\text{FC}(\text{Conv}(\mathbf{X}_t))))), \quad (6)$$

where  $\text{Conv}(\cdot)$  denotes a one-dimensional convolution operation with kernel size 3, and  $\text{FC}(\cdot)$  represents a fully connected layer. By stacking these convolutional layers followed by fully connected layers, the model effectively captures the temporal patterns embedded in the trend component. This hierarchical architecture enhances the model's representation capacity, allowing for more accurate reconstruction of the original time series, especially when subtle trend shifts carry significant information.

### E. Proximity-Enhanced Graph Convolutional Network

Conventional static graph structures are often inadequate for modeling time series data, as the underlying relationships may vary over time. To address this limitation, we propose an adaptive graph construction approach based on a gating network. This mechanism adaptively learns instance-specific graph topologies, enabling the model to more effectively capture temporal dynamics.

Traditional GCNs typically focus on capturing structural similarity between nodes. However, this approach may be insufficient for time series, which also exhibit strong temporal proximity, meaning that adjacent time steps often share similar characteristics. This temporal locality provides a valuable inductive bias for anomaly detection tasks. In our setting, the adaptive graphs are constructed over short sliding windows and are relatively dense and shallow; therefore, we adopt a standard Euclidean GCN, which is sufficient to model these local temporal dependencies while keeping the architecture and computation simple.

To leverage this property, we compute a prior based on the Laplacian kernel, which is derived solely from the raw temporal correlations of the input time series. This prior serves to guide the optimization of the GCN by encouraging it to focus on short-range temporal dependencies, which are essential for time series anomaly detection. By embedding the graph structure with the Laplacian kernel, we ensure that the model effectively captures the smooth temporal variations that are crucial for accurate anomaly detection. Before applying the graph module,

we project the stable component  $\mathbf{X}_s \in \mathbb{R}^{T \times N}$  into a higher-dimensional space using a linear transformation to enhance the model's representational capacity:

$$\mathbf{X}_l = \text{Linear}(\mathbf{X}_s) \in \mathbb{R}^{T \times d}, \quad (7)$$

where  $\mathbf{X}_s$  is the stable component and  $\text{Linear}(\cdot)$  denotes a learnable linear projection.

To obtain the adjacency matrix, we first introduce two learnable transformation matrices,  $\mathbf{V}_1 \in \mathbb{R}^{T \times k}$  and  $\mathbf{V}_2 \in \mathbb{R}^{k \times T}$ , which are designed to capture the underlying spatial dependencies within the multivariate time series, then we perform the following steps:

$$\mathbf{g}_1 = \text{NodeGate}_1([\mathbf{X}_l, \mathbf{V}_1]) \in \mathbb{R}^{T \times 1}, \quad (8)$$

$$\mathbf{g}_2 = \text{NodeGate}_2([\mathbf{X}_l, \mathbf{V}_2^\top]) \in \mathbb{R}^{T \times 1}, \quad (9)$$

$$\mathbf{P}_1 = \mathbf{g}_1 \odot \text{Linear}_1(\mathbf{X}_l) \in \mathbb{R}^{T \times k}, \quad (10)$$

$$\mathbf{P}_2 = \mathbf{g}_2 \odot \text{Linear}_2(\mathbf{X}_l) \in \mathbb{R}^{T \times k}, \quad (11)$$

$$\mathbf{V}'_1 = \mathbf{V}_1 + \mathbf{P}_1, \quad (12)$$

$$\mathbf{V}'_2 = \mathbf{V}_2 + \mathbf{P}_2^\top, \quad (13)$$

$$\mathbf{A} = \text{Softmax}(\mathbf{V}'_1 \mathbf{V}'_2) \in \mathbb{R}^{T \times T}. \quad (14)$$

We adopt an adaptive graph learning strategy that incorporates two learnable node representations,  $\mathbf{V}_1$  and  $\mathbf{V}_2$ . Here,  $k$  denotes the dimension of a latent space that is specifically designed to capture position-specific temporal semantics in a lightweight manner. These node matrices are concatenated with the projected input sequence  $\mathbf{X}_l$  and fed into two gating networks,  $\text{NodeGate}_1(\cdot)$  and  $\text{NodeGate}_2(\cdot)$ , which produce the gating values  $\mathbf{g}_1, \mathbf{g}_2$ . The gate outputs modulate the results of two independent linear transformations,  $\text{Linear}_1(\cdot)$  and  $\text{Linear}_2(\cdot)$ , yielding refinement terms  $\mathbf{P}_1$  and  $\mathbf{P}_2$ . These refinement terms are added to the node representations to obtain the updated matrices  $\mathbf{V}'_1 \in \mathbb{R}^{T \times k}$  and  $\mathbf{V}'_2 \in \mathbb{R}^{k \times T}$ . Finally, the adjacency matrix  $\mathbf{A}$  is computed from the row-wise softmax of the bilinear similarity  $\mathbf{V}'_1 \mathbf{V}'_2$ .

The Laplacian kernel-based temporal prior is defined as follows:

$$\sigma = \text{Linear}(\mathbf{X}_l) \in \mathbb{R}^{T \times 1}, \quad (15)$$

$$\mathbf{P} = \text{Softmax} \left( \left[ \frac{1}{\sigma_i} \exp \left( \frac{-|j-i|}{\sigma_i} \right) \right]_{i,j \in \{1, \dots, T\}} \right). \quad (16)$$

Here,  $\sigma \in \mathbb{R}^{T \times 1}$  is a vector of position-wise scaling factors generated by a learnable linear layer  $\text{Linear}(\cdot)$ , which maps each time step of  $\mathbf{X}_l$  to a scalar controlling both the amplitude and the decay width of the kernel. Using these scale values, a Laplacian kernel is constructed as  $\frac{1}{\sigma_i} \exp(-|j-i|/\sigma_i)$  for each temporal offset  $|j-i|$ , and a row-wise Softmax is then applied to normalize each row into a probability distribution, yielding the temporal prior  $\mathbf{P}$ .

We introduce a Laplacian prior consistency loss to encourage alignment between the learned graph structure and the prior

graph, defined as:

$$\mathcal{L}_{\text{prior}} = \text{KL}(\mathbf{P} \parallel \mathbf{A}) = \frac{1}{T} \sum_{i=1}^T \sum_{j=1}^T \mathbf{P}_{i,j} \ln \left( \frac{\mathbf{P}_{i,j}}{\mathbf{A}_{i,j}} \right). \quad (17)$$

Where  $\mathbf{A}_{i,j}$  and  $\mathbf{P}_{i,j}$  denote the elements in the  $i$ -th row and  $j$ -th column of the learned adjacency matrix  $\mathbf{A}$  and the Laplacian-based prior matrix  $\mathbf{P}$ , respectively. This loss term penalizes deviations between  $\mathbf{A}$  and  $\mathbf{P}$ , encouraging structural consistency. By minimizing the KL divergence, the model aligns its learned temporal connectivity with the prior distribution, thereby preserving temporal proximity in the learned graph. KL divergence is chosen because it directly compares row-wise probability distributions, is smooth and fully differentiable, and integrates naturally into gradient-based optimization. As a result,  $\mathcal{L}_{\text{prior}}$  serves as a useful inductive bias for time series anomaly detection by regularizing the learned graph towards plausible temporal structures without imposing a hard constraint.

To aggregate temporal features across different receptive fields, we employ a multi-order graph convolution module. Given the input sequence  $\mathbf{X}_l \in \mathbb{R}^{T \times d}$  and a set of adjacency matrices  $\mathbf{A}$ , the computation proceeds as follows:

$$\mathbf{H}^{(0)} = \mathbf{X}_l, \quad (18)$$

$$\mathbf{H}^{(1)} = \mathbf{A} \mathbf{H}^{(0)}, \quad (19)$$

$$\mathbf{H}^{(k)} = \mathbf{A} \mathbf{H}^{(k-1)}, \quad \forall k = 2, \dots, K, \quad (20)$$

$$\mathbf{H}_{\text{concat}} = \text{Concat}(\mathbf{H}^{(0)}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(K)}), \quad (21)$$

$$\mathbf{H}_g = \text{ReLU}(\text{MLP}(\mathbf{H}_{\text{concat}})). \quad (22)$$

This formulation aggregates temporal information from both short-range and long-range dependencies, enabling the model to capture multi-scale temporal dynamics essential for accurate anomaly detection. By iteratively applying the adjacency matrix and integrating multi-order representations, the model constructs expressive temporal embeddings while maintaining computational efficiency. Based on this framework, the corresponding algorithm is provided in Algorithm 1.

#### F. Mask-Optimized Multi-Head Attention Block

To better capture the structural characteristics of time series data, we introduce a mask tailored for anomaly detection into the attention matrix. Unlike standard attention mechanisms, which treat all time points equally, time series data typically exhibit locality bias, where adjacent time steps are more strongly correlated, while distant points may be irrelevant or noisy. To address this, we design a learnable mask based on the STE, which suppresses weak or spurious temporal dependencies and retains only the most informative connections. This masked attention mechanism improves the model's focus on salient patterns and reduces the risk of overfitting to noisy correlations, enhancing the robustness of anomaly detection.

We generate the attention mask from the input representation  $\mathbf{H}_g \in \mathbb{R}^{T \times d}$  as follows:

$$\mathbf{Z} = \text{Sigmoid}(\text{Linear}(\mathbf{H}_g)) \in \mathbb{R}^{T \times T}, \quad (23)$$

$$\mathbf{M} = \text{STE}(\mathbf{Z}) \in \mathbb{R}^{T \times T}. \quad (24)$$

**Algorithm 1:** Proximity-Enhanced Graph Convolutional Network.

---

```

1: Input: Time series  $\mathbf{X}_s \in \mathbb{R}^{T \times N}$ 
2: Initialize: Node representations
    $\mathbf{V}_1 \in \mathbb{R}^{T \times d}$ ,  $\mathbf{V}_2 \in \mathbb{R}^{d \times T}$ 
3: Compute: Linear transformation
    $\mathbf{X}_l = \text{Linear}(\mathbf{X}_s) \in \mathbb{R}^{T \times d}$ 
4: Compute: Gating values  $\mathbf{g}_1 =$ 
    $\text{NodeGate}_1([\mathbf{X}_l, \mathbf{V}_1])$ ,  $\mathbf{g}_2 = \text{NodeGate}_2([\mathbf{X}_l, \mathbf{V}_2^\top])$ 
5: Apply:
    $\mathbf{P}_1 = \mathbf{g}_1 \odot \text{Linear}_1(\mathbf{X}_l)$ ,  $\mathbf{P}_2 = \mathbf{g}_2 \odot \text{Linear}_2(\mathbf{X}_l)$ 
6: Update:  $\mathbf{V}'_1 = \mathbf{V}_1 + \mathbf{P}_1$ ,  $\mathbf{V}'_2 = \mathbf{V}_2 + \mathbf{P}_2^\top$ 
7: Compute: Adjacency matrix
    $\mathbf{A} = \text{Softmax}(\mathbf{V}'_1 \mathbf{V}'_2) \in \mathbb{R}^{T \times T}$ 
8: Compute Laplacian Kernel Temporal Prior:
9:    $\sigma = \text{Linear}(\mathbf{X}_l) \in \mathbb{R}^{T \times 1}$ 
10:   $\mathbf{P} = \text{Softmax}([\frac{1}{\sigma_i} \exp(\frac{-|j-i|}{\sigma_i})])_{i,j \in \{1, \dots, T\}}$ 
11: Compute Consistency Loss:  $\mathcal{L}_{\text{prior}} = \text{KL}(\mathbf{P} \parallel \mathbf{A})$ 
12: Multi-Order Graph Convolution:
13:   $\mathbf{H}^{(0)} = \mathbf{X}_l$ 
14:   $\mathbf{H}^{(1)} = \mathbf{A} \mathbf{H}^{(0)}$ 
15:  for  $k = 2$  to  $K$  do
16:     $\mathbf{H}^{(k)} = \mathbf{A} \mathbf{H}^{(k-1)}$ 
17:  end for
18: Aggregate:  $\mathbf{H}_{\text{concat}} = \text{Concat}(\mathbf{H}^{(0)}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(K)})$ 
19: Output:  $\mathbf{H}_g = \text{ReLU}(\text{MLP}(\mathbf{H}_{\text{concat}}))$ 

```

---

The input representation  $\mathbf{H}_g$  is first projected by a learnable linear layer  $\text{Linear}(\cdot)$  to produce pairwise logits in  $\mathbb{R}^{T \times T}$ . Applying  $\text{Sigmoid}(\cdot)$  element-wise yields the soft attention map  $\mathbf{Z}$ , where  $\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$ . The STE is then used to binarize  $\mathbf{Z}$  into a discrete mask  $\mathbf{M} \in \mathbb{R}^{T \times T}$  while preserving gradient flow during training. This mechanism enables the attention module to suppress weak or noisy temporal connections and emphasize informative dependencies in an end-to-end trainable manner.

To incorporate inductive bias favoring meaningful temporal associations, we apply the learned binary mask  $\mathbf{M}$  within the self-attention computation. This design enables the model to focus attention on relevant time steps while suppressing interactions that are likely to be noisy or spurious. Given the query, key, and value matrices  $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times d}$ , the masked attention mechanism is defined as:

$$\mathbf{T} = \mathbf{Q} \mathbf{K}^\top, \quad (25)$$

$$\mathbf{S} = \mathbf{T} \odot \mathbf{M} + (1 - \mathbf{M}) \odot (-\infty), \quad (26)$$

$$\text{MaskedScores} = \frac{\mathbf{S}}{\sqrt{d}}, \quad (27)$$

$$\text{Out} = \text{Softmax}(\text{MaskedScores}) \mathbf{V}. \quad (28)$$

This formulation ensures that only the temporal dependencies selected by the learned binary mask contribute to the final attention output. Specifically, positions with a mask value of 1 are preserved and influence the attention computation, while positions with a mask value of 0 are suppressed by assigning them

negligible attention weights. By combining hard binary masking with soft attention scoring, the model strikes a balance between interpretability and adaptability, improving its robustness in detecting temporal anomalies. We summarize this procedure as the masked attention function  $\text{MaskAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ .

Accordingly, we formulate the computation of our proposed Mask-optimized Multi-head Attention Block as follows:

$$\mathbf{Y} = \text{Conv}(\mathbf{H}_g), \quad (29)$$

$$\mathbf{Q}_i = \mathbf{Y} \mathbf{W}_i^Q, \quad \mathbf{K}_i = \mathbf{Y} \mathbf{W}_i^K, \quad \mathbf{V}_i = \mathbf{Y} \mathbf{W}_i^V, \quad i = 1, \dots, n, \quad (30)$$

$$\text{head}_i = \text{MaskAttention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i), \quad i = 1, \dots, n, \quad (31)$$

$$\mathbf{H}_s = \text{Concat}(\text{head}_1, \dots, \text{head}_n) \mathbf{W}^O. \quad (32)$$

Here,  $\mathbf{H}_g \in \mathbb{R}^{T \times d}$  denotes the input representation from the Proximity-enhanced Graph Convolutional Network. The  $\text{Conv}(\cdot)$  module applies a learnable one-dimensional convolution over the sequence dimension to produce an intermediate representation  $\mathbf{Y}$ , providing a local feature transformation prior to the attention computation.  $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V \in \mathbb{R}^{d \times d_h}$  are the learnable projection matrices for the  $i$ -th attention head, with  $d_h = d/n$  denoting the dimensionality per head. The corresponding query, key, and value matrices are  $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{T \times d_h}$ .  $\text{MaskAttention}(\cdot)$  refers to the masked attention mechanism introduced earlier. The outputs from all heads are concatenated into a matrix of shape  $\mathbb{R}^{T \times d}$  and projected using a shared matrix  $\mathbf{W}^O \in \mathbb{R}^{d \times d}$ . The final output  $\mathbf{H}_s \in \mathbb{R}^{T \times d}$  integrates multi-head attention outputs into a unified representation.

Finally, the reconstructed output  $\mathbf{X}_{\text{rec}} \in \mathbb{R}^{T \times N}$  is obtained by adding the trend-aware component  $\mathbf{H}_t \in \mathbb{R}^{T \times N}$ , which is derived from the Multi-layer Convolution Block, to a linearly projected representation of the stable component  $\mathbf{H}_s$ :

$$\mathbf{X}_{\text{rec}} = \mathbf{H}_s \mathbf{W}_{\text{rec}} + \mathbf{H}_t, \quad (33)$$

where  $\mathbf{W}_{\text{rec}} \in \mathbb{R}^{d \times N}$  is a learnable projection matrix.

This formulation allows the model to effectively integrate both the stable and trend components, enabling more accurate reconstruction of normal patterns in time series data. By preserving the contributions of long-term trends while emphasizing stable temporal dynamics, the model enhances its representational capacity, which is particularly beneficial for distinguishing subtle anomalies from normal fluctuations.

### G. Loss Function

The total loss consists of two components: a reconstruction loss defined as the mean squared error (MSE) between the input and the reconstructed output, and a prior alignment loss that promotes consistency between the learned adjacency matrix and the Laplacian-based temporal prior. The final loss function is defined as:

$$\mathcal{L}_{\text{total}} = \text{MSE}(\mathbf{X}, \mathbf{X}_{\text{rec}}) + \lambda \cdot \mathcal{L}_{\text{prior}}, \quad (34)$$

where  $\text{MSE}(\mathbf{X}, \mathbf{X}_{\text{rec}})$  measures the reconstruction quality,  $\mathcal{L}_{\text{prior}}$  denotes the Laplacian prior alignment loss, and  $\lambda$  controls the relative importance of the two terms.

### H. Anomaly Score

To compute the anomaly score, we apply the mean absolute error (MAE) between the input and the reconstructed sequence at each time step. This yields a point-wise anomaly score for the entire sequence. A threshold  $\delta$  is then applied: if the anomaly score at a specific time point exceeds  $\delta$ , it is classified as anomalous; otherwise, it is considered normal.

$$\text{Score}_i = \text{MAE}(\mathbf{X}_i, \mathbf{X}_{\text{rec},i}), \quad (35)$$

$$\mathcal{Y}_i = \begin{cases} 1, & \text{if } \text{Score}_i \geq \delta \\ 0, & \text{otherwise} \end{cases}. \quad (36)$$

### I. Time Complexity Analysis

To evaluate the computational efficiency of the proposed LaGraph model, we analyze the time complexity of its major components. The Expert Decomposition Block performs multiple moving average operations and weighted fusions, resulting in a time complexity of  $O(MTNr)$ , where  $M$  is the number of experts,  $T$  denotes the number of time steps,  $N$  is the number of variables (time series), and  $r$  denotes the kernel size of the moving-average operation, which can be implemented as a convolution with a fixed kernel. The Multi-layer Convolution Block involves stacked convolutional and fully connected layers, with a complexity of  $O(LkTN)$ , where  $L$  is the number of layers,  $k$  is the kernel size. The Proximity-enhanced Graph Convolutional Network dynamically constructs graphs based on the input and propagates temporal features through graph convolutions. This process incurs a time complexity of  $O(Td)$ , where  $d$  is the feature dimension. Although the parameters involved in the graph construction remain fixed during inference, the graph itself is still adaptively generated from the input. Therefore, the Proximity-enhanced Graph Convolutional Network contributes to the computational cost during inference. The Mask-optimized Multi-head Attention Block includes convolution, linear projections, and attention operations, resulting in a complexity of  $O(T^2d)$ . The final output reconstruction step has a complexity of  $O(TN)$ .

By aggregating all components, the total time complexity of LaGraph during training is:

$$O(MTNr + LkTN + T^2d + Td), \quad (37)$$

where  $M$ ,  $L$ ,  $r$ ,  $k$  and  $d$  are typically small constants.

During inference, although the graph structure is recomputed based on the input, the overall complexity remains tractable and is dominated by:

$$O(T^2d + LkTN), \quad (38)$$

demonstrating the model's efficiency for large-scale multivariate time series anomaly detection.

In our experiments, the sequence length  $T$  is at most around one hundred time steps, and the feature dimension  $d$  is of moderate size. Under this setting, the quadratic term  $O(T^2d)$  introduced by the Mask-optimized Multi-head Attention Block is computationally affordable, and the overall runtime and memory consumption are comparable to those of other attention-based time-series models used as baselines. Moreover, all other

TABLE I  
THE STATISTICS OF FIVE DATASETS

| Dataset    | #Channel | #Train  | #Test   | #Anomaly Rate |
|------------|----------|---------|---------|---------------|
| SMD        | 38       | 708,405 | 708,420 | 4.16%         |
| SWaT       | 51       | 47,515  | 44,986  | 11.97%        |
| MSL        | 55       | 58,317  | 73,729  | 5.88%         |
| Creditcard | 29       | 142,403 | 142,404 | 0.17%         |
| GECCO      | 9        | 69,260  | 69,261  | 1.25%         |

“#” indicates number of.

components of LaGraph scale linearly with respect to  $T$ . For extremely long sequences, we acknowledge that the quadratic attention term may become a potential bottleneck, and exploring more efficient attention variants represents an interesting direction for future work.

## IV. EXPERIMENTS

### A. Datasets and Settings

1) *Datasets*: We evaluate our method on five widely used real-world datasets: SMD, SWaT, MSL, Creditcard (NIPS\_TS\_Creditcard), and GECCO (NIPS\_TS\_GECCO). The statistics of these datasets are described in Table I.

- **SMD (Server Machine Dataset)** is a publicly available multivariate time series dataset designed for anomaly detection in server monitoring scenarios. Collected and released by Su et al. [55], the dataset originates from the monitoring logs of 28 servers in a large Internet company. Each server is monitored across 38 features, including CPU load, memory usage, and network traffic, with data sampled at one-minute intervals. The dataset spans a continuous 10-week period, with the first 5 weeks used as the training set and the remaining 5 weeks as the test set. Anomalies in the test set, along with the affected features, were labeled by domain experts based on actual incident reports. The entire dataset contains approximately 1.4 million observations, with an anomaly ratio of about 4.16%. The SMD dataset serves as a critical benchmark for evaluating and developing multivariate time series anomaly detection methods.
- **SWaT (Secure Water Treatment)** [56] is a widely used benchmark for research on the security of industrial control systems. Collected from a testbed that simulates a water treatment plant, the dataset covers a six-stage process, from raw water storage to reverse osmosis. It records a variety of sensor and actuator data, including flow rates, water levels, and chemical properties. The data were collected over 11 consecutive days, with the first 7 days representing normal, attack-free operation and the final 4 days containing various intentional attacks on the system. This dataset is commonly used for training and evaluating intrusion detection models, contributing to the development of effective security mechanisms that enhance the resilience of industrial control systems.
- **MSL (Mars Science Laboratory)** [57] is a real-world, expert-labeled dataset of spacecraft telemetry collected



from the Curiosity rover during its mission on Mars. The dataset consists of multivariate time series data from 55 telemetry channels, capturing various aspects of the rover's operations and environment. It contains approximately 66,709 data points, annotated with 36 distinct anomaly sequences identified by mission experts through Incident Surprise, Anomaly (ISA) reports. These anomalies reflect actual issues encountered during rover activities, including both point anomalies and contextual anomalies that depend on the temporal context of the data. The MSL dataset serves as a valuable benchmark for evaluating anomaly detection methods in complex, dynamic, and high-stakes aerospace scenarios.

- **Creditcard (NIPS\_TS\_Creditcard)** [58] is a real-world, expert-labeled dataset of credit card transactions collected by a European bank over two days in September 2013. The dataset contains multivariate time series and tabular data representing 284,807 transactions, with 28 anonymized principal components derived via PCA, along with time and amount features. Among these, only 492 transactions are labeled as fraudulent, resulting in a highly imbalanced dataset with an anomaly ratio of approximately 0.172%. The anomalies correspond to actual fraud cases encountered in real-world banking operations and include both point anomalies and context-dependent anomalies. This dataset serves as a valuable benchmark for evaluating anomaly detection and imbalanced classification methods in complex, high-stakes financial scenarios.
- **GECCO (NIPS\_TS\_GECCO)** [58] is a real-world, expert-labeled dataset collected from an IoT-based drinking water quality monitoring system. Originally released as part of the GECCO 2018 Industrial Challenge, the dataset contains multivariate time series data recording various water quality metrics, such as chemical properties and flow measurements, to monitor and ensure the safety of drinking water. It includes both normal and anomalous patterns, with anomalies reflecting real-world events that deviate from expected operational behaviors. The GECCO dataset serves as a valuable benchmark for evaluating anomaly detection methods, particularly in IoT and environmental monitoring scenarios, where pattern-wise outliers are prevalent.

2) *Baselines*: We compare our method with 12 popular models, including CATCH [45], Timer-XL [59], MTST [60], iTransformer [19], DLinear [61], ModernTCN [62], TimesNet [63], Peri-mid Former [18], TSINR [64], Dcdetector [25], PatchTST [46], and Anomaly Transformer [24].

- **CATCH (2025)**: A frequency patching framework for multivariate time series anomaly detection is proposed. It divides the frequency domain into patches to capture fine-grained features and utilizes a Channel Fusion Module with masked attention to learn dynamic channel correlations. The framework is guided by a bi-level multi-objective optimization approach.
- **Timer-XL (2025)**: A unified time series forecasting model is introduced that reformulates multivariate forecasting as next-token prediction. Using a causal Transformer with a universal TimeAttention mechanism and temporal

position embeddings, the method captures fine-grained dependencies across long contexts and achieves state-of-the-art performance, including strong zero-shot results.

- **MTST (2024)**: A multi-resolution Transformer is introduced that models temporal patterns at different frequencies using patch segments of varying lengths and relative positional encoding. This design enables simultaneous learning of short-term fluctuations and long-term seasonalities, achieving state-of-the-art forecasting performance.
- **iTransformer (2024)**: A Transformer-based time series forecasting model is proposed, which embeds time points as variate tokens and applies attention across variates to capture multivariate correlations. The model employs feed-forward networks for each variate token to learn nonlinear representations without altering the core Transformer components, achieving strong performance and scalability.
- **DLinear (2023)**: A simple time series forecasting model is proposed, which decomposes the series into trend and remainder components, using separate linear networks for direct multi-step prediction. It outperforms complex Transformer-based models by focusing on non-autoregressive forecasting, rather than relying on the Transformer's temporal relation extraction.
- **ModernTCN (2024)**: A convolution-based time series model is proposed that modernizes and adapts the traditional TCN for time series tasks. It achieves state-of-the-art performance across multiple tasks by expanding the effective receptive field, balancing efficiency and accuracy more effectively than Transformer- and MLP-based models.
- **TimesNet (2022)**: A general-purpose time series model is proposed that transforms 1D sequences into 2D tensors to capture intra- and inter-period variations using 2D kernels. Its TimesBlock adaptively discovers multi-periodicity and models complex temporal patterns efficiently, achieving state-of-the-art performance across diverse time series tasks.
- **Peri-mid Former (2024)**: A time series model is proposed that decomposes multi-periodic variations into a hierarchical periodic pyramid and applies self-attention to capture inclusion, overlap, and adjacency relations among periodic components, achieving state-of-the-art performance across diverse time series tasks.
- **TSINR (2024)**: A time series anomaly detection framework, TSINR, leverages large language models to enhance feature representations and employs implicit neural representations to decompose signals into trend, seasonal, and residual components, achieving state-of-the-art performance on standard anomaly detection benchmarks.
- **DCdetector (2023)**: A time series anomaly detection model is proposed that learns discriminative, permutation-invariant representations through multi-scale dual attention and contrastive learning. By creating a permuted environment and optimizing with pure contrastive loss, the model achieves state-of-the-art performance on benchmark datasets.
- **PatchTST (2022)**: A Transformer-based model for multivariate time series forecasting and self-supervised learning



is proposed. It segments the series into subseries patches as tokens and uses channel-independent weights. This design preserves local semantics, reduces computation, and enables longer dependencies, achieving state-of-the-art performance in both forecasting and transfer learning.

- *Anomaly Transformer (2021)*: An unsupervised time series anomaly detection model is proposed that leverages self-attention to measure association discrepancies between time points. By detecting anomalies through biased local associations and enhancing distinguishability via a min-max strategy, the model achieves state-of-the-art performance across diverse benchmarks.

All baselines use the configurations from their original paper or the official code to ensure a fair comparison.

3) *Implementation Details*: For model training and evaluation, we apply a sliding window with a stride of 1 to generate diverse input pairs from the dataset. The training, validation, and test sets are then normalized using the mean and standard deviation computed from the training set.

Our experiments are implemented in PyTorch and run on a single NVIDIA RTX 4090 24 GB GPU. Model optimization is performed using the Adam optimizer with a learning rate of  $1 \times 10^{-4}$ .

By default, the LaGraph framework adopts an input embedding dimension of 256, transforming raw time series data into a higher-dimensional latent space to enhance representational capacity. The encoder consists of three stacked layers, designed to capture hierarchical temporal dependencies through sequential feature transformation. A multi-head attention mechanism with four parallel heads is employed to enable the model to attend to diverse temporal patterns by aggregating information from multiple subspaces. Additionally, the GCN is implemented with three layers, facilitating the progressive integration of local and high-order temporal dependencies via the learned graph structure.

For each dataset, we tune the main hyperparameters using a simple grid search over a small candidate set. Concretely, the sliding-window size is selected from the range [36, 132] with a step of 12, and the model dimension  $d_{\text{model}}$  is chosen from {64, 128, 256, 512} based on validation performance. This limited search space keeps the tuning overhead manageable, while allowing LaGraph to adapt to different temporal scales and dataset characteristics and achieve robust performance across various benchmarks.

4) *Metrics*: To evaluate model performance, we adopt precision, recall, and F1 score as evaluation metrics. Traditional metrics, while effective for discrete tasks, may not fully capture the nuanced nature of continuous anomaly patterns in time series data. A widely used alternative is the point adjusted F1 score [65], which considers detecting a single anomaly within a continuous segment as detecting all anomalies in that segment. However, this approach can lead to misleading results and performance overestimation.

To address this limitation, recent studies have adopted the affiliated F1 score [66], a more fine-grained metric that calculates precision based on the average directional distance from predicted anomaly events to their corresponding ground truth

events, while recall is defined based on the average directional distance from ground truth events to predicted anomalies. This metric provides a more convincing evaluation of anomaly detection performance in the context of continuous time series data, and is employed throughout our experiments.

## B. Experimental Results

We evaluated our proposed method against a range of state-of-the-art baselines on five widely used benchmark datasets: SMD, SWaT, MSL, GECCO, and Creditcard. The detailed performance comparison in terms of Precision, Recall, and F1-score is shown in Table II. The results demonstrate that our method achieves the highest F1-score across all datasets, effectively balancing precision and recall to validate its superior comprehensive performance in time series anomaly detection.

Specifically, on the SMD dataset, our method achieved an F1-score of 0.8495, outperforming the next best baseline (CATCH) by 0.21%. On the SWaT dataset, our model reached an F1-score of 0.8771, exceeding the second-best method (ModernTCN) by 14.95%. For the MSL dataset, our model achieved a competitive F1-score of 0.7302, slightly surpassing PatchTST and DLinear. On the GECCO dataset, our model achieved the highest F1-score of 0.9388, confirming its robustness in complex multivariate time series. Notably, on the Creditcard dataset, our method delivered an F1-score of 0.7546, outperforming all competing baselines.

These consistent improvements across multiple datasets demonstrate the comprehensive effectiveness and superiority of our method in managing the inherent trade-off between false alarms and missed detections.

## C. Ablation Study

To assess the contribution of each architectural component in our proposed framework LaGraph, we conduct comprehensive ablation studies involving its four principal modules: the Expert Decomposition Block, the Multi-layer Convolution Block, the Proximity-enhanced Graph Convolutional Network, and the Mask-optimized Multi-head Attention Block. For the Proximity-enhanced GCN, we evaluate its impact by first removing the Laplacian kernel prior while retaining the graph convolution operations, and then by entirely eliminating the GCN module. Similarly, for the Mask-optimized Multi-head Attention Block, we assess its effect by removing the learnable attention mask, followed by the complete removal of the attention mechanism. These controlled experimental settings enable a systematic evaluation of each module's contribution to anomaly detection performance.

From Table III, it is evident that removing any individual component from LaGraph results in a performance decline across all datasets, confirming the effectiveness of each module. Among them, the Multi-layer Convolution Block and the Expert Decomposition Block have the most substantial impact, as their removal leads to the largest reductions in F1 scores, particularly on the SMD and SWaT datasets, respectively. In contrast, removing the learnable mask causes the smallest performance degradation across most datasets, indicating their impact is weaker than other

TABLE II  
THE EXPERIMENTAL RESULTS ON FIVE DATASETS, WHERE P, R, AND F1 REPRESENT PRECISION, RECALL, AND F1 SCORE, RESPECTIVELY. **BOLD** INDICATES THE BEST RESULTS, WHILE UNDERLINED INDICATES THE SECOND-BEST RESULTS.

| Method     |    | Ours          | CATCH         | T-XL          | MTST   | iTrans        | DLin          | Moder         | Tsnet         | Peri          | TSINR         | DC     | Patch  | Atrans        |
|------------|----|---------------|---------------|---------------|--------|---------------|---------------|---------------|---------------|---------------|---------------|--------|--------|---------------|
| SMD        | P  | 0.7895        | 0.7728        | 0.6537        | 0.7268 | 0.7363        | 0.7611        | 0.7543        | <u>0.8269</u> | 0.8191        | <b>0.8325</b> | 0.5031 | 0.7474 | 0.6279        |
|            | R  | 0.9193        | 0.9379        | <b>0.9719</b> | 0.7961 | 0.9432        | 0.9399        | 0.9474        | 0.7260        | 0.6946        | 0.6459        | 0.8952 | 0.9698 | 0.8931        |
|            | F1 | <b>0.8495</b> | <u>0.8474</u> | 0.7817        | 0.7599 | 0.8270        | 0.8411        | 0.8399        | 0.7732        | 0.7517        | 0.7274        | 0.6442 | 0.8442 | 0.7374        |
| SWaT       | P  | <b>0.8270</b> | 0.5551        | 0.5508        | 0.5507 | 0.5606        | 0.5517        | 0.5871        | <u>0.6786</u> | 0.6331        | 0.5552        | 0.5327 | 0.5676 | 0.5375        |
|            | R  | 0.9336        | 0.9694        | 0.9154        | 0.9414 | 0.9687        | 0.8645        | 0.9563        | 0.3839        | 0.3223        | <b>0.9960</b> | 0.9807 | 0.9566 | <u>0.9923</u> |
|            | F1 | <b>0.8771</b> | 0.7060        | 0.6878        | 0.6949 | 0.7102        | 0.6735        | <u>0.7276</u> | 0.4903        | 0.4271        | 0.7130        | 0.6904 | 0.7124 | 0.6973        |
| MSL        | P  | 0.5927        | 0.5924        | 0.5430        | 0.5521 | 0.5661        | 0.5769        | 0.5781        | 0.6314        | <b>0.6501</b> | <u>0.6324</u> | 0.5759 | 0.5842 | 0.5354        |
|            | R  | 0.9507        | 0.7597        | <b>0.9795</b> | 0.8819 | 0.9504        | <u>0.9755</u> | 0.9752        | 0.4028        | 0.3560        | 0.3990        | 0.8738 | 0.9517 | 0.8658        |
|            | F1 | <b>0.7302</b> | 0.6657        | 0.6987        | 0.6791 | 0.7095        | 0.7250        | <u>0.7259</u> | 0.4919        | 0.4601        | 0.4893        | 0.6942 | 0.7240 | 0.6616        |
| Creditcard | P  | 0.6299        | 0.6176        | 0.5538        | 0.5635 | 0.5586        | 0.6045        | 0.6105        | 0.6895        | <u>0.7209</u> | <b>0.7261</b> | 0.4841 | 0.6113 | 0.5150        |
|            | R  | 0.9408        | 0.9558        | 0.8768        | 0.8286 | <b>0.9845</b> | 0.9485        | 0.9517        | 0.7198        | 0.6545        | 0.5798        | 0.8946 | 0.9529 | 0.8443        |
|            | F1 | <b>0.7546</b> | <u>0.7503</u> | 0.6788        | 0.6708 | 0.7127        | 0.7384        | 0.7439        | 0.7043        | 0.6861        | 0.6448        | 0.6282 | 0.7448 | 0.6398        |
| GECCO      | P  | <b>0.8882</b> | 0.8320        | 0.7078        | 0.7014 | 0.7348        | 0.8081        | 0.8079        | <u>0.8522</u> | 0.8165        | 0.7799        | 0.4943 | 0.8314 | 0.6861        |
|            | R  | 0.9954        | <b>0.9984</b> | 0.9236        | 0.9445 | 0.9787        | 0.9975        | 0.9977        | 0.6571        | 0.7383        | 0.2567        | 0.8648 | 0.9954 | 0.8608        |
|            | F1 | <b>0.9388</b> | <u>0.9076</u> | 0.8014        | 0.8050 | 0.8394        | 0.8928        | 0.8928        | 0.7420        | 0.7754        | 0.3863        | 0.6291 | 0.9060 | 0.7636        |

TABLE III  
ABLATION STUDY ON THREE DATASETS. **BOLD** INDICATES THE BEST RESULTS.

| Method         | SMD           |               |               | SWaT          |               |               | MSL           |               |               |
|----------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                | P             | R             | F1            | P             | R             | F1            | P             | R             | F1            |
| w/o Decomp     | 0.7465        | <b>0.9458</b> | 0.8344        | 0.5944        | <b>0.9855</b> | 0.7415        | 0.5662        | 0.9416        | 0.7072        |
| w/o Multi_conv | 0.7622        | 0.8480        | 0.8028        | 0.6520        | 0.9433        | 0.7710        | 0.5410        | 0.9175        | 0.6807        |
| w/o Laplace    | 0.7870        | 0.9055        | 0.8421        | 0.8165        | 0.9338        | 0.8712        | 0.5656        | 0.9239        | 0.7016        |
| w/o GCN        | 0.7876        | 0.8889        | 0.8352        | 0.7613        | 0.9192        | 0.8328        | 0.5680        | 0.9235        | 0.7034        |
| w/o mask       | 0.7761        | 0.9113        | 0.8383        | 0.8217        | 0.9342        | 0.8744        | 0.5671        | 0.9402        | 0.7075        |
| w/o attention  | <b>0.7948</b> | 0.9011        | 0.8446        | 0.7833        | 0.8966        | 0.8362        | 0.5590        | 0.9323        | 0.6989        |
| ours           | 0.7895        | 0.9193        | <b>0.8495</b> | <b>0.8270</b> | 0.9336        | <b>0.8771</b> | <b>0.5927</b> | <b>0.9507</b> | <b>0.7302</b> |

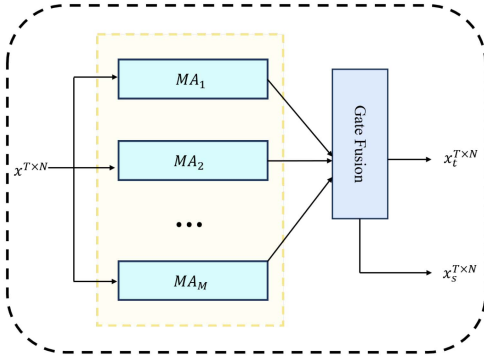
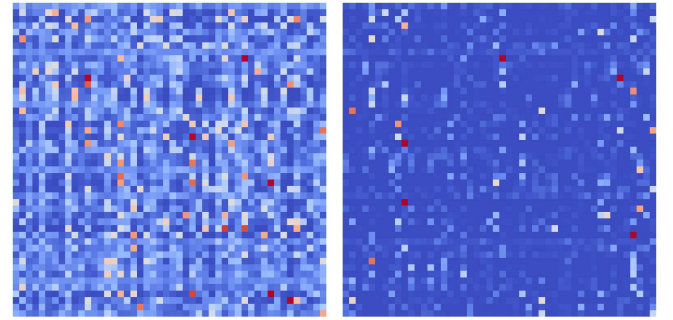


Fig. 2. The framework of Expert Decomposition Block is illustrated.

components. These findings underscore the importance of the full model architecture and the complementary roles of each module in achieving optimal anomaly detection performance.

In addition, we study the effect of integrating the Laplacian-kernel prior into the proposed framework on the MSL dataset. To this end, we analyze how the adjacency-matrix weights change with and without the prior. Fig. 3 shows that introducing the Laplacian prior noticeably amplifies the weights near the diagonal, indicating a stronger emphasis on temporal proximity. These observations suggest that the proposed prior helps the model better capture local temporal dependencies in the learned graph structure, thereby improving both the interpretability and the empirical performance of the Proximity-enhanced GCN.

Moreover, to evaluate the effectiveness of different temporal prior configurations in our LaGraph framework, we conduct an ablation study on three datasets, comparing the proposed Laplacian+KL prior with other combinations such as Gaussian+KL



(a) Original Model

(b) W/o Prior

Fig. 3. Visualization of the learned adjacency matrices on the MSL dataset: (a) Original Model, (b) W/o Prior.

TABLE IV  
ABLATION STUDY ON THREE DATASETS WITH DIFFERENT PRIORS: F1 SCORE COMPARISON. **BOLD** INDICATES THE BEST RESULTS.

| Method              | SMD           | SWaT          | MSL           |
|---------------------|---------------|---------------|---------------|
| w/o prior           | 0.8421        | 0.8712        | 0.7016        |
| Gaussian+KL         | 0.8422        | 0.8716        | 0.7268        |
| Laplacian+ $\ell_2$ | 0.8419        | 0.8690        | 0.6891        |
| Gaussian+ $\ell_2$  | 0.8296        | 0.8613        | 0.6967        |
| (Laplacian+KL) Ours | <b>0.8495</b> | <b>0.8771</b> | <b>0.7302</b> |

and Laplacian+ $\ell_2$ . As shown in Table IV, the Laplacian+KL configuration consistently achieves the best performance across all datasets, obtaining F1 scores of 0.8495 on SMD, 0.8771 on SWaT, and 0.7302 on MSL. These results indicate that combining the Laplacian kernel with a KL divergence regularization term provides a particularly effective inductive bias for time series anomaly detection.

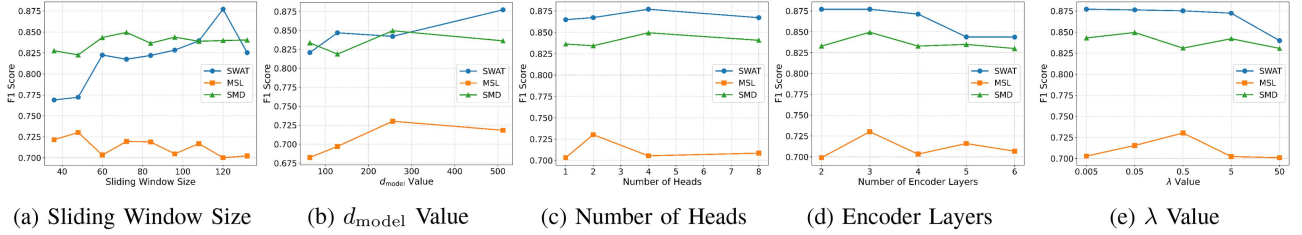


Fig. 4. Parameter Sensitivity Analysis on three datasets SWaT, MSL and SMD. (a) shows the trend of F1 scores as the sliding window size changes. (b) shows the trend of F1 scores as the dimension of the stable component ( $d_{\text{model}}$ ), after passing through a linear layer, changes. (c) shows the trend of F1 scores as the number of heads changes. (d) shows the trend of F1 scores as number of Encoder Layers change. (e) shows the trend of F1 scores as  $\lambda$  Value change.

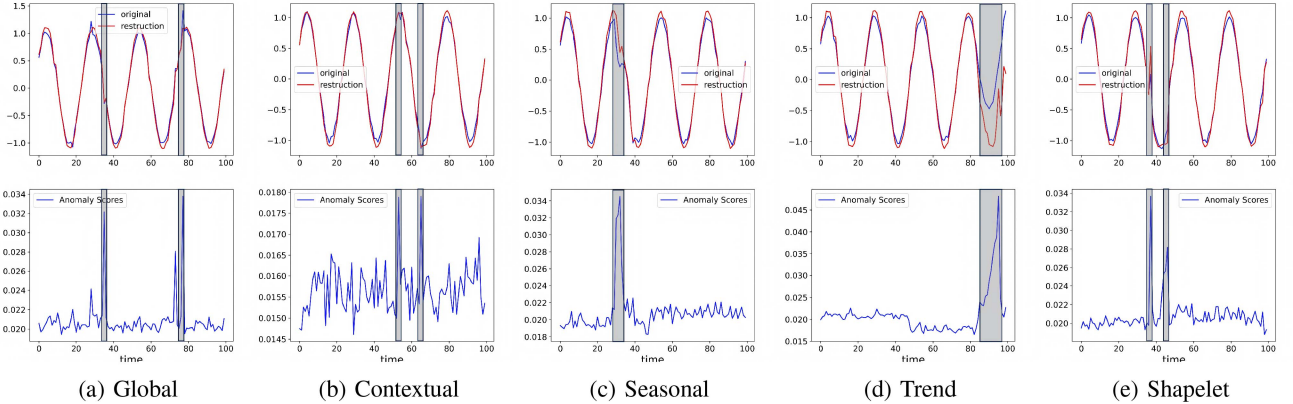


Fig. 5. Visualization comparison of the original input, reconstructed result, and anomaly scores across different anomaly categories. The first row in each sub-figure compares the original input with the reconstructed result, while the second row displays the corresponding anomaly scores for global, contextual, seasonal, trend, and shapelet anomalies.

#### D. Parameter Sensitivity Analysis

There are four important hyperparameters in the proposed method, LaGraph: Sliding Window Size,  $d_{\text{model}}$  (dimension of the stable component after passing through a linear layer), the number of heads in the attention module, the number of encoder layers, and  $\lambda$  (the weighting coefficient of the prior loss). We conduct experiments on three commonly used datasets: SWaT, MSL, and SMD to analyze the model performance as these parameters vary, as shown in Fig. 4.

From Fig. 4(a), we observe significant changes in the F1 score with variations in the sliding window size. The optimal sliding window size differs for each dataset, indicating that the model's performance is sensitive to this parameter. Selecting an appropriate sliding window size is crucial, as it enables the model to effectively capture temporal dependencies and trends.

Fig. 4(b) shows the trend of F1 scores as the dimension of the stable component,  $d_{\text{model}}$ , changes. The F1 score fluctuates as  $d_{\text{model}}$  increases, suggesting that this parameter significantly impacts performance. Tuning  $d_{\text{model}}$  helps balance model complexity and detection accuracy, particularly for large datasets.

From Fig. 4(c), we see that the F1 score varies slightly as the number of heads in the attention module increases. Although the F1 score fluctuates, the trend is not drastic, indicating that the number of heads impacts model performance, but the effect is less pronounced than for other parameters. The optimal number of heads can be selected to balance performance and computational efficiency.

Fig. 4(d) shows that increasing the number of encoder layers initially improves performance, but after a certain point, the F1 score starts to decrease. This suggests that while additional layers can enhance performance, the gains diminish and eventually lead to performance degradation. To balance performance and efficiency, we set the default number of encoder layers to 3, which yields optimal results without overfitting.

Fig. 4(e) shows that the optimal value of  $\lambda$  varies across different datasets. Specifically, the F1 score for MSL peaks at  $\lambda = 0.5$ , while SMD achieves its best performance at  $\lambda = 0.05$ . SWaT exhibits higher robustness, maintaining stable results across a broader range of  $\lambda$ . Consequently, the value of  $\lambda$  is determined individually for each dataset, reflecting the varying importance of structural priors across different temporal dynamics.

#### E. Visual Analysis

Fig. 5 demonstrates a detailed comparison of the original input, the reconstructed results, and the anomaly scores across different anomaly categories [58]. In this figure, the first row of each sub-figure compares the original input with the reconstructed result, while the second row displays the corresponding anomaly scores for global, contextual, seasonal, trend, and shapelet anomalies. The image clearly marks the locations of anomalies, with higher anomaly scores corresponding to these marked points, indicating significant deviations from expected patterns. The reconstructed signals exhibit a smoother profile compared to the original inputs, suggesting the model's effective



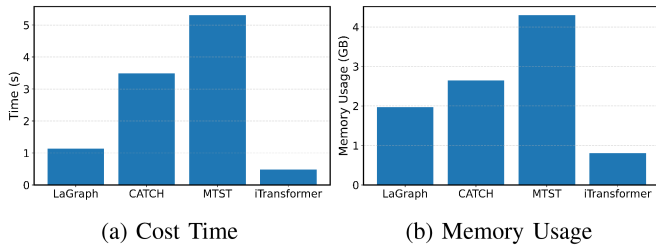


Fig. 6. Visualization comparison of the efficiency of the four models on the SMD dataset in terms of runtime and memory usage. (a) reports the cost time of each model per 100 iterations, while (b) presents their corresponding memory usage.

filtering of noise while preserving underlying temporal trends. This smoothing effect is particularly noticeable during periods of high fluctuation in the original data. The marked anomalies, where the original data shows greater deviations, are associated with significantly higher anomaly scores, further validating the model's ability to detect substantial anomalies. These findings highlight the robustness of the LaGraph model for anomaly detection in time series analysis.

#### F. Efficiency Analysis

On the SMD dataset, Fig. 6 compares LaGraph with three representative baselines (CATCH, MTST, and iTransformer) in terms of runtime and memory consumption. It should be noted that the primary objective of LaGraph is to improve anomaly detection accuracy, particularly in terms of the F1-score, rather than to optimize computational efficiency alone. This efficiency analysis is therefore conducted to assess the practical feasibility of the proposed method. For fairness, all methods are evaluated with the same batch size and sliding-window size, under the same hardware and implementation settings. As shown in the figure, while LaGraph is not the most efficient model overall, its overhead remains well controlled: it is clearly faster than MTST and falls within the range of the other baselines. In terms of memory usage, LaGraph also exhibits stable and moderate consumption, with a lighter footprint than MTST, which incurs the highest cost. Overall, although LaGraph does not achieve the best efficiency on every metric, its time and space overheads remain practical for deployment, especially considering its superior anomaly detection performance.

#### V. CONCLUSION

This paper proposed a novel model **LaGraph**, for time series anomaly detection. To better preserve key temporal patterns and reduce the interference of anomalies in reconstruction-based frameworks, we introduced the Expert Decomposition Block, which separates the input series into stable and trend components. This design facilitates a more accurate representation of temporal dynamics, which in turn enhances the reliability and robustness of the reconstruction. To capture temporal proximity, we designed the Proximity-enhanced Graph Convolutional Network, which leverages a Laplacian kernel to emphasize relationships between adjacent time points. In addition, we proposed the Mask-optimized Multi-head Attention Block to adaptively

suppress weak or noisy graph connections and improve feature fusion. Extensive experiments on five real-world datasets verified the effectiveness and superiority of the proposed method **LaGraph**.

#### REFERENCES

- [1] D. Fährmann, L. Martín, L. Sánchez, and N. Damer, "Anomaly detection in smart environments: A comprehensive survey," *IEEE Access*, vol. 12, pp. 64006–64049, 2024.
- [2] X. Yang, X. Qi, and X. Zhou, "Deep learning technologies for time series anomaly detection in healthcare: A review," *IEEE Access*, vol. 11, pp. 117788–117799, 2023.
- [3] H. Wu, H. Zhou, M. Long, and J. Wang, "Interpretable weather forecasting for worldwide stations with a unified deep model," *Nature Mach. Intell.*, vol. 5, no. 6, pp. 602–611, 2023.
- [4] W. Zhang et al., "Irregular traffic time series forecasting based on asynchronous spatio-temporal graph convolutional networks," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 4302–4313.
- [5] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–33, Apr. 2021, doi:10.1145/3444690.
- [6] M. Gupta, J. Gao, C. C. Aggarwal, and J. Han, "Outlier detection for temporal data: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 9, pp. 2250–2267, Sep. 2014.
- [7] R. Tsay and A. Pankratz, "Outliers in multivariate time series," *Biometrika*, vol. 87, pp. 789–804, Dec. 2000.
- [8] M. A. Belay, S. S. Blakseth, A. Rasheed, and P. Salvo Rossi, "Unsupervised anomaly detection for IoT-based multivariate time series: Existing solutions, performance analysis and future directions," *Sensors*, vol. 23, no. 5, 2023, Art. no. 2844.
- [9] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," *ACM Comput. Surv.*, vol. 57, no. 1, pp. 1–42, Oct. 2024, doi:10.1145/3691338.
- [10] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [11] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: John Wiley Sons, Inc., 2015.
- [12] M. G. Kendall, "The advanced theory of statistics," vol. 23, no. 1, p. 310, 1961.
- [13] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.
- [14] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
- [15] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
- [16] E. Fix, *Discriminatory Analysis: Nonparametric Discrimination, Consistency Properties*, vol. 1. Wright-Patterson Air Force Base, OH, USA: USAF School Aviation Med., 1985.
- [17] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," vol. 35, no. 5, pp. 4027–4035, 2021.
- [18] Q. Wu, G. Yao, Z. Feng, and Y. Shuyuan, "Peri-midFormer: Periodic pyramid transformer for time series analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 13035–13073.
- [19] Y. Liu et al., "iTransformer: Inverted transformers are effective for time series forecasting," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 4004–4028.
- [20] C. Wang et al., "Drift doesn't matter: Dynamic decomposition with diffusion reconstruction for unstable multivariate time series anomaly detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 10758–10774.
- [21] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 3072–3090.
- [22] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [23] W. Yue et al., "Sub-adjacent transformer: Improving time series anomaly detection with reconstruction error from sub-adjacent neighborhoods," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 2524–2532.



- [24] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly Transformer: Time series anomaly detection with association discrepancy," in *Proc. 10th Int. Conf. Learn. Representations*, 2022, pp. 5203–5222.
- [25] Y. Yang, C. Zhang, T. Zhou, Q. Wen, and L. Sun, "DCdetector: Dual attention contrastive representation learning for time series anomaly detection," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 3033–3045.
- [26] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [27] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [28] A. Graves, "Long short-term memory," in *Supervised Sequence Labelling With Recurrent Neural Networks*. Berlin, Germany: Springer, 2012, pp. 37–45.
- [29] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 2713–2726.
- [31] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 2920–2931.
- [32] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1024–1034.
- [33] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 9104–9120.
- [34] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10466–10485, Dec. 2024.
- [35] C. Ding, S. Sun, and J. Zhao, "MST-GAT: A multimodal spatial-temporal graph attention network for time series anomaly detection," *Inf. Fusion*, vol. 89, pp. 527–536, 2023.
- [36] W. Zhang, C. Zhang, and F. Tsung, "GRLeN: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 2390–2397.
- [37] S. Wang, Y. Zhang, X. Lin, Y. Hu, Q. Huang, and B. Yin, "SAGoG: Similarity-aware graph of graphs neural networks for multivariate time series classification," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 8, pp. 4820–4832, Aug. 2025.
- [38] M. Han and Q. Wang, "Adaptive graph convolution neural differential equation for spatio-temporal time series prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 6, pp. 3193–3204, Jun. 2025.
- [39] B. Zheng et al., "Adversarial graph neural network for multivariate time series anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 7612–7626, Dec. 2024.
- [40] Y. Zheng et al., "Correlation-aware spatial-temporal graph learning for multivariate time-series anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11802–11816, Sep. 2024.
- [41] X. Huang, W. Chen, B. Hu, and Z. Mao, "Graph mixture of experts and memory-augmented routers for multivariate time series anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 16, pp. 17476–17484.
- [42] L. Sun, J. Ye, H. Peng, and P. S. Yu, "A self-supervised Riemannian GNN with time varying curvature for temporal graph learning," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1827–1836.
- [43] L. Sun et al., "Hyperbolic variational graph neural network for modeling dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 5, pp. 4375–4383.
- [44] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. H. Hoi, "ETSFormer: Exponential smoothing transformers for time-series forecasting," 2022, *arXiv:2202.01381*.
- [45] X. Wu et al., "CATCH: Channel-aware multivariate time series anomaly detection via frequency patching," in *Proc. 13th Int. Conf. on Learn. Representations*, 2025, pp. 56894–56922.
- [46] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 33132–33155.
- [47] S. Feng et al., "Multi-scale attention flow for probabilistic time series forecasting," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 5, pp. 2056–2068, May 2024.
- [48] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, and R. Jin, "Make transformer great again for time series forecasting: Channel aligned robust dual transformer," 2023, *arXiv:2305.12095*.
- [49] X. Wang, T. Zhou, Q. Wen, J. Gao, B. Ding, and R. Jin, "CARD: Channel aligned robust blend transformer for time series forecasting," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 20577–20615.
- [50] D. Du, B. Su, and Z. Wei, "Preformer: Predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2023, pp. 1–5.
- [51] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "TSLANet: Re-thinking transformers for time series representation learning," 2024, *arXiv:2404.08472*.
- [52] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 12, pp. 11106–11115.
- [53] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," *Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 22419–22430.
- [54] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 27268–27286.
- [55] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
- [56] A. P. Mathur and N. O. Tippenhauer, "Swat: A water treatment testbed for research and training on ICS security," in *Proc. Int. Workshop Cyber-Phys. Syst. Smart Water Netw.*, 2016, pp. 31–36.
- [57] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 387–395.
- [58] K.-H. Lai, D. Zha, J. Xu, Y. Zhao, G. Wang, and X. Hu, "Revisiting time series outlier detection: Definitions and benchmarks," in *Proc. 35th Conf. Neural Inf. Process. Syst. Track Datasets Benchmarks*, 2021, pp. 1868–1884.
- [59] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "Timer-XL: Long-context transformers for unified time series forecasting," in *Proc. 13th Int. Conf. Learn. Representations*, 2025, pp. 51892–51916.
- [60] Y. Zhang, L. Ma, S. Pal, Y. Zhang, and M. Coates, "Multi-resolution time-series transformer for long-term forecasting," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2024, pp. 4222–4230.
- [61] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 9, pp. 11121–11128.
- [62] D. Luo and X. Wang, "ModernTCN: A modern pure convolution structure for general time series analysis," in *Proc. 12th Int. Conf. Learn. Representations*, 2024, pp. 34187–34208.
- [63] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *Proc. 11th Int. Conf. Learn. Representations*, 2023, pp. 6423–6445.
- [64] M. Li, K. Liu, H. Chen, J. Bu, H. Wang, and H. Wang, "TSINR: Capturing temporal continuity via implicit neural representations for time series anomaly detection," 2024, *arXiv:2411.11641*.
- [65] H. Xu et al., "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications," in *Proc. World Wide Web Conf.*, 2018, pp. 187–196.
- [66] A. Huet, J. M. Navarro, and D. Rossi, "Local evaluation of time series anomaly detection algorithms," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 635–645.



**Shicong Zeng** received the BS degree in computer science and technology from Central South University, Changsha, China. He is currently working toward the MS degree with the Department of Computer Science and Technology, Harbin Institute of Technology, Weihai, China. His research interests include deep learning, data mining, and time series analysis.



**Guoqing Chao** received the PhD degree with the Department of Computer Science and Technology, East China Normal University, Shanghai, China, in 2015. From 2015 to 2020, he was a postdoc with the University of Connecticut and Northwestern University, US and Singapore Management University, Singapore. He is currently with the Harbin Institute of Technology, Weihai, China. He has authored or coauthored more than 40 papers. His research interests include machine learning, data mining and bioinformatics.

He is a reviewer of many prestigious journals such as *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *JMLR*, *IEEE Transactions on Image Processing*, *IEEE Transactions on Neural Networks and Learning Systems*, and *IEEE Transactions on Knowledge and Data Engineering*. He is on the Editorial Board member of *Machine Learning and Knowledge Extraction* and leading guest editor of Special issues in several prestigious journals such as *Applied Intelligence* and *Neural Processing Letters*.



**Zhijin Wang** (Member, IEEE) received the PhD degree from the Department of Computer Science and Technology, East China Normal University, Shanghai, China, in 2016. He is currently an associate professor with Jimei University, Xiamen, China. He is also the founder of Forecasting and Time Series (FAST) Community and leader of the pyFAST library for time series analysis. His research interests include recommendation systems, time series forecasting, and artificial intelligence in health and healthcare.



**Junquan Wei** is currently working toward the BS degree in artificial intelligence from the Harbin Institute of Technology, Weihai, China. His research interests include time series forecasting and time series anomaly detection.



**Dianhui Chu** received the PhD degree with the School of Computer Science and Technology, Harbin Institute of Technology, Harbin, China in 2014. He is currently the Shandong Taishan scholar. He has authored or coauthored more than 50 papers. His research interests include service computing and software service engineering, cloud computing and mobile internet technology, data mining and Big Data analysis



**Yanwei Yu** (Member, IEEE) received the PhD degree in computer science from the University of Science and Technology Beijing, Beijing, China, in 2014. From 2012 to 2013, he was a visiting scholar with Worcester Polytechnic Institute. From 2016 to 2018, he was a postdoctoral researcher with the College of Information Sciences and Technology, Pennsylvania State University. He is currently a professor with the College of Computer Science and Technology, Ocean University of China. His research interests include data mining and machine learning.